

Logica di Base

Mario Bravetti

`mario.bravetti@unibo.it`

Ingegneria e Scienze Informatiche, Cesena
ALMA MATER STUDIORUM—Università di Bologna





Introduction

Propositional Logic

Deduction Systems

Predicate Logic



Goals

Understanding:

- the general framework of logic
- propositional and predicate (first-order) logics
- deductive systems

Later in this course we will see:

- the role of logics in computational models
- what is model checking



What is logic and why using it

Teoria fondamentale che sta alla base di gran parte della matematica e dell'informatica.

It is a foundational theory

Costruita sulla Teoria degli Insiemi

- Built on top of set theory

La Logica e la Teoria degli Insiemi sono strettamente interconnesse. Molti concetti logici hanno un'analogia diretta nelle operazioni tra insiemi.

Può essere usata per definire la Teoria degli Insiemi

- Can itself be used to define set theory

La logica è il metalinguaggio che definisce le regole di base per costruire tutta la matematica, compresa la Teoria degli Insiemi.

What it is used for

Una Proposizione è una frase dichiarativa che può essere solamente Vera o Falsa. I Connettivi Logici definiscono le relazioni tra queste proposizioni, permettendoci di costruire formule complesse e di calcolarne il valore di verità.

La Logica esprime Proposizioni e Relazioni tra esse

- A logic expresses propositions and relations between them

- (deep relation with philosophy)

La logica formale è nata dalla filosofia, in particolare con Aristotele. Si occupa dell'analisi della struttura del ragionamento corretto e della validità delle argomentazioni, che sono questioni filosofiche fondamentali.

In Computer Science

La logica è la spina dorsale dell'informatica moderna, dato che i computer sono macchine logiche che operano su stati binari

La logica è usata per specificare esattamente cosa deve fare un sistema:

- Verifica di Programmi: Si usano formule logiche per esprimere la preconditione (cosa deve essere vero prima di eseguire un codice) e la postcondizione (cosa sarà vero dopo l'esecuzione del codice).
- Specifiche Formali: Nei sistemi critici (software di controllo aereo, centrali nucleari), la logica è usata per specificare senza ambiguità tutti i requisiti.

Fornisce un Linguaggio per Esprimere Proprietà

- Provides a language for expressing properties of interest of a system

Fornisce un Modo per Attuare queste Proprietà nel Sistema

- Provides a way of enacting these properties into the system

La logica si traduce direttamente nella struttura fisica e operativa del computer:

- Circuiti Digitali/Porte Logiche: Alla base di ogni microprocessore ci sono le Porte Logiche (AND, OR, NOT), che sono la realizzazione fisica dei connettivi logici. Il valore Vero è l'alta tensione, Falso è la bassa tensione.
- Programmazione Logica: Linguaggi come Prolog utilizzano la logica dei predicati come base. Il programma è una collezione di fatti e regole logiche, e il computer usa l'inferenza logica per rispondere alle query.
- Database: La logica è fondamentale per le interrogazioni e la gestione dei dati.



Basic elements of a logic

Syntax

La Sintassi definisce come scrivere una formula che sia corretta (ben formata) all'interno del sistema logico. Riguarda solo la forma e le regole di costruzione, ignorando il significato.

A set of admissible *formulas* $\mathcal{F} = \{\phi_0, \phi_1, \dots\}$

Sono l'insieme di tutte le espressioni che seguono le regole grammaticali della logica.

- Typically defined by a context-free grammar

Semantics

La Semantica definisce il significato di una formula. Stabilisce come interpretare la formula e se essa è Vera o Falsa.

Defined by two elements $\langle \mathcal{M}, \models \rangle$

- $\mathcal{M}$ is a set of *models* ($M \in \mathcal{M}$)
- $\models \subset \mathcal{M} \times \mathcal{F}$ is the *satisfaction* relation

Un Modello (M) è un'interpretazione o una valutazione che assegna un valore di verità ai simboli di una formula.

La Relazione di Soddisfazione è la regola che definisce quando un modello M rende vera una formula phi.

Terminology and notation:

- $M \models \phi$ means: model M satisfies formula ϕ
- (or also: formula ϕ is satisfied in interpretation/valuation M)
- Also write $M \models \phi, \phi', \dots$ for: $M \models \phi$ and $M \models \phi'$ and $\dots$

Si legge "M soddisfa phi" oppure "phi è vera nel modello M."

Una notazione abbreviata per dire che lo stesso modello M rende vere contemporaneamente più formule



Propositional Logic

It is the most basic logic introduced

Syntax of a formula ϕ

La Sintassi ci dice quali sequenze di simboli sono formule ben formate e quali no. Definisce la grammatica del nostro linguaggio.

Rappresentano le affermazioni elementari a cui possiamo assegnare un valore di verità (Vero o Falso).

proposizioni atomiche.

Given a finite set of propositional symbols ($P = \{p, q, r, \dots\}$), a formula can be:

- $\top$ (true constant) Sempre vera, indipendentemente da tutto
- $\perp$ (false constant) Sempre falsa, indipendentemente da tutto.
- a propositional symbol $p \in P$ Proposizione atomica: l'elemento di base
- of the kind $\neg\phi$ (informal meaning: not ϕ) Inverte il valore di verità di
- of the kind $\phi \vee \phi'$ (informal meaning: ϕ or ϕ') Vera se almeno una delle due è vera.
- of the kind $\phi \wedge \phi'$ (informal meaning: ϕ and ϕ') Vera solo se entrambe sono vere.
- of the kind $\phi \rightarrow \phi'$ (informal meaning: ϕ implies ϕ') Se phi è vera, allora anche phi' lo è. Spesso letta "Se phi, allora phi"

nothing else can be a formula ϕ .



Propositional Logic

More formally, using context-free grammars.

Per definire la Sintassi della Logica Proposizionale in modo non ambiguo si usa la notazione tipica della Context-Free Grammar.

Syntax

Questa definizione è ricorsiva: le formule complesse sono costruite a partire da formule più semplici.

Una formula phi può essere

- Grammar: $\phi ::= p \mid \top \mid \perp \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \rightarrow \phi$
- p is a *proposition* symbol ($p \in P = \{p, q, r, \dots\}$)
- Example: $p \wedge (q \vee r) \rightarrow \neg p$

Model/Interpretation/Valuation

Dopo aver stabilito la forma (Sintassi), passiamo al significato (Semantica). Il concetto fondamentale della Semantica in PL è il Modello (M).

- A model M is a function $M : P \mapsto \{F, T\}$
- It associates a boolean value to each proposition

Modello (M): Non è altro che un modo specifico di assegnare i valori di verità a tutte le proposizioni atomiche P.

M è una funzione matematica:

- Dominio (P): L'insieme di tutti i simboli proposizionali ($\{p, q, r, \dots\}$).
- Codominio (F,T): L'insieme dei due valori booleani: Falso (False) e Vero (True).

Il modello M fa l'unica cosa che conta nella logica proposizionale: stabilisce lo stato di verità del mondo che stiamo analizzando.



Propositional Logic

Semantics

Truth tables for logical connectives:

ϕ	ψ	$\phi \wedge \psi$	ϕ	ψ	$\phi \vee \psi$
T	T	T	T	T	T
T	F	F	T	F	T
F	T	F	F	T	T
F	F	F	F	F	F

ϕ	ψ	$\phi \rightarrow \psi$	ϕ	$\neg \phi$
T	T	T	T	F
T	F	F	F	T
F	T	T		
F	F	T		

Propositional Logic

Semantics

la Semantica è l'insieme di regole che, dato un Modello (M), stabilisce se una formula phi è Soddisfatta (Vera) in quel modello. La relazione chiave è $M \models \phi$. La definizione è ricorsiva: prima si definiscono le proposizioni atomiche, poi si usano i connettivi per definire le formule complesse.

Definition of satisfaction relation $M \models \phi$

Regole
Base

- $M \models p \iff M(p) = \mathbf{T}$ La proposizione atomica p è vera nel modello M se e solo se la valutazione M le ha assegnato il valore True (Vero).
- $M \models \mathbf{T}$ Questa costante è sempre soddisfatta, in qualsiasi modello M.
- $M \models \perp$ does not hold (written $M \not\models \perp$) Questa costante non è mai soddisfatta (è sempre falsa), in nessun modello M.
- $M \models \neg\phi \iff \text{not } M \models \phi$ (i.e. $M \not\models \phi$) è vera se e solo se phi non è vera nel modello M. (NOT)
- $M \models \phi \wedge \phi' \iff M \models \phi \text{ and } M \models \phi'$ è vera se e solo se entrambe le formule, phi e phi', sono vere. (AND)
- $M \models \phi \vee \phi' \iff M \models \phi \text{ or } M \models \phi'$ è vera se e solo se almeno una tra phi e phi' è vera. (OR)
- $M \models \phi \rightarrow \phi' \iff M \models \phi \text{ implies } M \models \phi'$ è vera in tutti i casi, tranne quando phi (l'antecedente) è vera e phi' (il conseguente) è falsa. (IMPLICA)

that is application of *truth tables* for “not”, “and”, “or”, “implies”.

Example

- $\{p \mapsto F, q \mapsto T\} \models \mathbf{T}, \neg\perp, q, \neg p, q \vee p, q \wedge \neg p, p \rightarrow q$



On Propositional Logic

Scope

la Logica Proposizionale è essenziale perché fissa le regole del gioco: è il sistema più semplice che ci permette di definire il concetto di validità logica e inferenza usando un linguaggio formale e non ambiguo.

La Logica Proposizionale è il punto di partenza per tutti i sistemi logici più complessi

- It is the basic logic upon which all others are built
- It is the framework where standard concepts are formally defined (true, false, or, and, ...)

La Logica Proposizionale traduce il linguaggio naturale ambiguo in termini matematici rigorosi, stabilendo una volta per tutte cosa significano concetti come "vero", "falso" e "implica"



Some general concepts about logics (1)

Logical Consequence

- Definition: Formula ϕ is *logical consequence* of $\phi_1, \dots, \phi_n$ if, for any M , when $M \models \phi_1, \dots, \phi_n$ then also $M \models \phi$
- This can be expressed by notation: $\phi_1, \dots, \phi_n \models \phi$
- Examples in Propositional Logic: $p, q \models p \wedge q, p \vee q$
 $p \wedge q \models p \vee q$

Tautology/Validity

- Definition: A formula ϕ that is logical consequence of no formulas (i.e. it is always true in any model) is called a *tautology*
- Notation: $\models \phi$ (we also say that ϕ is *valid*)
- Examples in Propositional Logic: $\models \top, \neg \perp, p \rightarrow p, p \vee \neg p$



Some general concepts about logics (2)

More complex examples of logical consequence

- $\alpha \models \beta \rightarrow \alpha$
- $\alpha \rightarrow (\beta \rightarrow \gamma) \models (\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma)$
- $\neg\alpha \rightarrow \neg\beta \models \beta \rightarrow \alpha$

Proof by contradiction

If $\phi_1, \dots, \phi_n, \neg\phi \models \perp$ then $\phi_1, \dots, \phi_n \models \phi$

An exercise

What is the logical consequence of contradictory premises?

- $\phi, \neg\phi \models \alpha$, which α ?
that is, for any M it must hold:
whenever $M \models \phi, \neg\phi$ then also $M \models \alpha$



Some general concepts about logics (3)

Logical Equivalence

- Definition: Formula ϕ and ϕ' are said *logically equivalent* (written $\phi \equiv \phi'$) if $\phi \models \phi'$ and $\phi' \models \phi$

Examples in Propositional Logic

- De Morgan Law: $\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$
- $p \rightarrow q \equiv (\neg p) \vee q \equiv (\neg q) \rightarrow (\neg p)$



Some general concepts about logics (4)

Satisfiability

- Definition: Formula ϕ is *satisfiable* if $M \models \phi$ for some M
- Examples in Propositional Logic:
 - $\top, p, p \vee q$ are satisfiable
 - $\neg\top, \perp, p \wedge \neg p$ are *not* satisfiable.

On relationship between satisfiability and validity

ϕ is valid if and only if $\neg\phi$ is not satisfiable



Some general concepts about logics (5)

On Logical Consequence

$\phi_1, \dots, \phi_n \models \phi$ if and only if $\models (\phi_1 \wedge \dots \wedge \phi_n) \rightarrow \phi$

On Logical Equivalence (LE)

$\phi \equiv \phi'$ if and only if $\models (\phi \rightarrow \phi') \wedge (\phi' \rightarrow \phi)$

- Operator $\leftrightarrow$ is sometimes added to mean LE ($\dots \models \phi \leftrightarrow \phi'$)

A note

These facts do not hold in general, since not all logics may have operator $\rightarrow$, however they hold in all logics extending Propositional Logic, that is, on all standard logics



A simplified version of Propositional Logic

Some equivalence

All operators can be defined in terms of $\neg$ and $\rightarrow$ only:

$$\begin{aligned}\top &\equiv p \rightarrow p \\ \perp &\equiv \neg(p \rightarrow p) \\ \phi \vee \phi' &\equiv (\neg\phi) \rightarrow \phi' \\ \phi \wedge \phi' &\equiv \neg(\phi \rightarrow \neg\phi')\end{aligned}$$

Syntax

$$\phi ::= p \mid \neg\phi \mid \phi \rightarrow \phi$$

(Semantics is the simplified version of what we saw before)

(BTW: Another simplified syntax is $\phi ::= p \mid \perp \mid \phi \rightarrow \phi$)



Some general concepts about logics (6)

Computing Logical Equivalence and Tautology check

How can we know that $\phi \models \phi'$?

- This should hold for any model M , which are infinite
- Fortunately, in PL, meaningful M are those using variables in ϕ and ϕ' only (which are finite)
- Basically, it is about building the truth table of ϕ and ϕ'

Similar argument holds for checking whether $\models \phi$

The importance of “logical consequence”

- If we are able to show that $\phi_1 \dots \phi_n \models \phi \dots$
- .. it means we know in one shot that ϕ holds for a large set of models (those in which $\phi_1 \dots \phi_n$ hold)



A problem with validity

Motivation

Given a logic $\langle \mathcal{F}, \models \rangle$:

- Suppose we want to check whether $\phi \in \mathcal{F}$ is valid, that is $\models \phi$
- Suppose the definition of relation $\models$ does not suggest an effective way of computing it (for Propositional Logic we do not have this problem, for other Logics we *do* have it)
- We may need a procedure to check validity, e.g., using a mechanism resembling simplification of mathematical expressions (a *calculus*) or theorem proving in mathematics



An example in Propositional Logic

Even in Propositional Logic we can make an example

- Suppose we want to check
$$\models \neg((a \wedge b \wedge c \wedge d \wedge e) \rightarrow c) \rightarrow (f \vee g \vee h \vee i)$$
- It amounts to build a tableau of $2^9 = 512$ rows (it could be worse than that)

Resorting to CNF

A more efficient technique is to turn the formula into CNF (Conjunctive Normal Form):

- a conjunction ($\wedge$) of clauses
- a clause is a disjunction ($\vee$) of literals
- a literal is a (positive literal) or $\neg a$ (negative literal) for some a

We just check that each clause contains a pair of opposite literals.



Deduction Systems

Validity can be “deduced” by the following proof-like reasoning

- $(a \wedge b \wedge c \wedge d \wedge e) \rightarrow c$ is true since on left we have a conjunction including the implied formula
- $\neg((a \wedge b \wedge c \wedge d \wedge e) \rightarrow c)$ is false
- a false formula implies any formula

We used “axioms”: $\phi \wedge \phi' \rightarrow \phi$, $\neg\neg\top$, $\perp \rightarrow \phi$

The notion of Logical Deduction

A *deduction* relation $\vdash$ is introduced:

- $\phi_1, \dots, \phi_n \vdash \phi$ means ϕ can be deduced from $\phi_1, \dots, \phi_n$
- in particular $\vdash \phi$ means ϕ can be deduced from no formula
- ($\models$ is called semantic entailment, $\vdash$ syntactic entailment)



Axioms and rules

Defining a deduction system

While relation $\models$ can be defined in any mathematical way, relation $\vdash$ is always defined constructively, by *axioms* and *rules*

- Form of axioms: $\vdash \phi$
- Form of rules:
$$\frac{\vdash \phi_1 \dots \vdash \phi_n}{\vdash \phi}$$

(it means: ϕ holds if all preconditions $\phi_1, \dots, \phi_n$ hold)

where $\phi, \phi_1, \dots$ are formulas or formula schemas (formulas with variables, that can be substituted by any formula)



Hilbert deduction system

Deduction system

Minimal syntax: $\phi ::= p \mid \neg\phi \mid \phi \rightarrow \phi$

Axioms:

- $\vdash \alpha \rightarrow (\beta \rightarrow \alpha)$
- $\vdash (\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$
- $\vdash (\neg\alpha \rightarrow \neg\beta) \rightarrow (\beta \rightarrow \alpha)$

Rule:

- $$\frac{\vdash \alpha \quad \vdash \alpha \rightarrow \beta}{\vdash \beta} \quad (\text{Modus Ponens})$$

Note: parenthesis are not part of the language, they are just a way to group sub-expressions



Derivation of a deduction

Derivation of a deduction

Proving $\phi_1 \dots \phi_n \vdash \phi$ is done by proving $\frac{\vdash \phi_1 \dots \vdash \phi_n}{\vdash \phi}$, that is by deriving $\vdash \phi$ under the assumptions $\vdash \phi_1 \dots \vdash \phi_n$

Deduction system

$$A1 \vdash \alpha \rightarrow (\beta \rightarrow \alpha)$$

$$A2 \vdash (\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$$

$$A3 \vdash (\neg \alpha \rightarrow \neg \beta) \rightarrow (\beta \rightarrow \alpha)$$

$$\text{MP} \frac{\vdash \alpha \quad \vdash \alpha \rightarrow \beta}{\vdash \beta} \quad (\text{Modus Ponens})$$

We prove $\alpha \rightarrow \gamma, \gamma \rightarrow \beta \vdash \alpha \rightarrow \beta$ that is $\frac{\vdash \alpha \rightarrow \gamma \quad \vdash \gamma \rightarrow \beta}{\vdash \alpha \rightarrow \beta}$

$$\begin{array}{c} \vdash \alpha \rightarrow \gamma \quad [MP] \quad \frac{[MP] \frac{\vdash \gamma \rightarrow \beta \quad [A1] \frac{\vdash (\gamma \rightarrow \beta) \rightarrow (\alpha \rightarrow (\gamma \rightarrow \beta))}{\vdash \alpha \rightarrow (\gamma \rightarrow \beta)}}{[A2] \frac{\vdash (\alpha \rightarrow (\gamma \rightarrow \beta)) \rightarrow ((\alpha \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta))}{\vdash (\alpha \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta)}}}{[MP] \frac{\vdash (\alpha \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta)}{\vdash \alpha \rightarrow \beta}} \end{array}$$



Soundness and completeness of Hilbert deduction system

Soundness of $\vdash$ with respect to $\models$

If $\phi_1, \dots, \phi_n \vdash \phi$ then $\phi_1, \dots, \phi_n \models \phi$

That is: derived deductions are indeed logical consequences!

Completeness of $\vdash$ with respect to $\models$

If $\phi_1, \dots, \phi_n \models \phi$ then $\phi_1, \dots, \phi_n \vdash \phi$

All deductions that are logical consequences can be derived!



Decidability of Hilbert deduction system

Decidability of Hilbert deduction system

There is an algorithm that establishes whether a derivation for $\phi_1 \dots \phi_n \vdash \phi$, i.e. for $\frac{\vdash \phi_1 \dots \vdash \phi_n}{\vdash \phi}$, exists (by using axioms and rules and instantiating their variables), and computes such a derivation.



Deduction systems are pure-syntax!

The "philosophy" of deduction systems..

- They give a “typographical” way of checking validity of a formula
- You do not need to know the meaning of $\neg$, $\rightarrow$, ...
- Those axioms and rules tell you what is such meaning



Predicate Logic

Propositional Logic

You can model sentences like:

- If it rains, then it is true that it rains or it snows!
- $p = \text{it rains}, q = \text{it snows}$
- $\models p \rightarrow p \vee q$

Predicate (First-Order) Logic

Predicates should be more structured, and discuss individual and global properties

- All men are mortal, Socrates is a men, therefore Socrates is mortal
- ...



Predicate Logic

Example 1

- All men are mortal, Socrates is a men.
Therefore Socrates is mortal
- $\models (\forall X (man(X) \rightarrow mort(X)) \wedge man(socrat)) \rightarrow mort(socrat)$

Example 2

- Every student is younger than some instructor
- $M \models (\forall X (stud(X) \rightarrow (\exists Y (instr(Y) \wedge younger(X, Y)))))$



Predicate Logic – Ingredients

Examples

- $\models (\forall X (man(X) \rightarrow mort(X)) \wedge man(socrat)) \rightarrow mort(socrat)$
- $M \models (\forall X (stud(X) \rightarrow (\exists Y (instr(Y) \wedge younger(X, Y))))$

Ingredients w.r.t. Propositional Logic

- Universal ($\forall$) and existential ($\exists$) quantification
- atomic formulas are not just propositional symbols $p, q, \dots$
- but are predicates applied to terms ($man/1, younger/2, \dots$)
- terms can be variables (X) or constants ($socrat$)



Predicate Logic – Interpretation

Examples

- $\models (\forall X (man(X) \rightarrow mort(X)) \wedge man(socrat)) \rightarrow mort(socrat)$
- $M \models (\forall X (stud(X) \rightarrow (\exists Y (instr(Y) \wedge younger(X, Y))))$

How interpreting 1L

- Note that first example above is a tautology independently of the meaning of *man/1*, *mort/1*, and constant *socrat*, that is, independently from the model *M*.
- The second example is true only if *M* is properly introduced that defines how *stud/1*, *instr/1* and *younger/2* work.
- *M* should provide an interpretation for predicates and terms!



Syntax

Terms

A term t over (C, F) has syntax: $t ::= x \mid c \mid f(t_1, \dots, t_n)$

- a variable $(x, y, z, \dots)$
- a constant $(c, d, e, \dots \in C)$
- a compound term: an n -ary function $(f \in F)$ with t_i terms as arguments

a term yields an individual in the domain of discourse!



Syntax

Formulas

A formula ϕ over (C, F, P) has syntax:

$$\phi ::= p(t_1, \dots, t_n) \mid \top \mid \perp \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \rightarrow \phi \mid \forall x\phi \mid \exists x\phi$$

- an atomic formula: an n -ary predicate ($p \in P$) with t_i terms over (C, F) as arguments (n can be 0)
- a formula $\top$, $\perp$, $\neg\phi$, $\phi \wedge \phi$, $\phi \vee \phi$, $\phi \rightarrow \phi$ (as for propositions)
- a universal quantification of x in ϕ
- an existential quantification of x in ϕ



Interpretation

Structure of a model

A model M for formulas over (C, F, P) is of the kind $\langle A, \mathcal{C}, \mathcal{F}, \mathcal{P} \rangle$

- A is a non-empty universe of concrete values (the individuals)
- $\mathcal{C} : C \mapsto A$ associates to each constant a concrete value
- $\mathcal{F} : F \mapsto (A^n \mapsto A)$ associates to each function name a concrete function
- $\mathcal{P} : P \mapsto (A^n \mapsto \{F, T\})$ associates to each predicate name a concrete predicate

Understanding

That is, a specific model M associates:

- a term t (once its variables have been substituted with concrete values) to a concrete value
- an atomic formula $p(t_1, \dots, t_n)$ to a boolean value



An example of model: students and instructor

$$M \models (\forall X(stud(X) \rightarrow (\exists Y(instr(Y) \wedge younger(X, Y)))))$$

$$C = \{\}, F = \{\}, P = \{stud/1, instr/1, younger/2\}$$

$$\text{Model } M = \langle A, \mathcal{C}, \mathcal{F}, \mathcal{P} \rangle$$

- $A = \{\text{bravetti}, \text{rossi}, \text{omicini}, \text{bianchi}, \text{natali}\}$
- $\mathcal{C} = \{\}, \mathcal{F} = \{\}$
- $\mathcal{P} = \{$
 - $stud(\text{bravetti})=F, stud(\text{rossi})=T, stud(\text{omicini})=F,$
 $stud(\text{bianchi})=T, stud(\text{natali})=F$
 - $instr(\text{bravetti})=T, instr(\text{rossi})=F, instr(\text{omicini})=T,$
 $instr(\text{bianchi})=F, instr(\text{natali})=T$
 - $younger(\text{rossi}, \text{natali})=T, younger(\text{natali}, \text{omicini})=F,$
 $younger(\text{bianchi}, \text{bravetti})=T, \dots$ $\}$



An example of model: M_P , Peano arithmetics

Formulas

- Constant z
- Functions $s/1$, $+/2$, $*/2$ (in infix notation)
- Predicates $>/2$, $=/2$ (in infix notation)

The model $\langle A, \mathcal{C}, \mathcal{F}, \mathcal{P} \rangle$

- A : the set $\mathbb{N} = \{0, 1, 2, \dots\}$ of natural numbers
- $\mathcal{C}$ associates constant z to 0
- $\mathcal{F}$ associates function name $s/1$ to the function that computes the successor, $+/2$ to mathematical sum (e.g. $s(z) + s(z)$ is associated to 2), ...
- $\mathcal{P}$ associates predicate name $>/2$ to the greater than predicate between numbers



Example of formulas

Do these satisfaction relations hold?

Natural numbers are not closed under square root:

- $M_P \models \exists y \neg \exists x (x * x = y)$

Existence of a greater value:

- $M_P \models \forall x \exists y (y > x)$

What is the following about?

- $M_P \models \forall q \exists p (p > q \wedge \forall x \forall y ((x > s(z) \wedge y > s(z)) \rightarrow \neg (x * y = p)))$



Closed Formulas

Free and bound variables

- The occurrence of a variable x in formula ϕ is said to be *bound*, if that occurrence is inside a *binder* of kind $\forall x$ or $\exists x$
- The occurrence of a variable x in formula ϕ is said to be *free* if it is not *bound*
- Example: in $p(x) \wedge \forall x(p(x) \vee c)$ the left-most occurrence of x is free, the right-most occurrence of x is bound

Closed Formulas

ϕ is a closed formula if it does not include free variable occurrences

We will consider closed formulas only, unless stated otherwise.



Semantics of predicate logic

Semantics (for closed formulas)

Definition of satisfaction relation $M \models \phi$

- $M \models p(t_1, \dots, t_n) \iff M(p(t_1, \dots, t_n)) = \text{T}$
- $M \models \top, M \not\models \perp$
- $M \models \neg\phi \iff M \not\models \phi$
- $M \models \phi \wedge \phi' \iff M \models \phi \text{ and } M \models \phi'$
- $M \models \phi \vee \phi' \iff M \models \phi \text{ or } M \models \phi'$
- $M \models \phi \rightarrow \phi' \iff M \models \phi \text{ implies } M \models \phi'$
- $M \models \forall x\phi \iff \text{for all } v \in A \text{ we have } M \models \phi[v/x]$
- $M \models \exists x\phi \iff \text{there exists } v \in A \text{ such that } M \models \phi[v/x]$

where $\phi[v/x]$ means substituting *free occurrences* of x in ϕ with v

Note: definition of $M \models \phi$ is based on checking $M \models \phi'$ for (sub)formulas ϕ' including values v instead of free variables x



Semantics of predicate logic

Semantics: formal definition of $M(p(t_1, \dots, t_n))$

Remember: model $M = \langle A, \mathcal{C}, \mathcal{F}, \mathcal{P} \rangle$

$M()$ over atomic formulas

- $M(p(t_1, \dots, t_n)) = \mathcal{P}(p)(M(t_1), \dots, M(t_n))$

$M()$ over terms t (with variables x substituted by values v)

- $M(v) = v$
- $M(c) = \mathcal{C}(c)$
- $M(f(t_1, \dots, t_n)) = \mathcal{F}(f)(M(t_1), \dots, M(t_n))$



Logical Consequence/Equivalence, Validity/Satisfiability

Same definitions as for Propositional Logic

Example of logical equivalence

- $\exists x\phi \equiv \neg\forall x\neg\phi$

A simplified version of Predicate Logic

Syntax: $\phi ::= p(t_1, \dots, t_n) \mid \neg\phi \mid \phi \rightarrow \phi \mid \forall x\phi$

(Semantics is the simplified version of what we saw before)



Validity in predicate logic is undecidable!!

Undecidability of the validity problem for predicate logic

Turing and Church independently proved that:

- The problem $\models \phi$ is in general undecidable (over formulas ϕ using equality and another predicate with arity at least 2)

Semi-decidability of the validity problem for predicate logic

A procedure implementing $\models \phi$, for a general ϕ , exists such that:

- if $\models \phi$, a positive answer is provided in a finite amount of time
- if $\not\models \phi$, the procedure does not terminate



Hilbert deduction system

Deduction system

Minimal syntax: $\phi ::= p(t_1, \dots, t_n) \mid \neg\phi \mid \phi \rightarrow \phi \mid \forall x\phi$

Axioms:

- .. those of Propositional Logic.. and
- $\vdash (\forall x\alpha) \rightarrow \alpha[t/x]$ with no free variable in t becoming bound
- $\vdash \forall x(\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \forall x\beta)$ with x not free in α

Rules:

- $$\frac{\vdash \alpha \quad \vdash \alpha \rightarrow \beta}{\vdash \beta} \quad (\text{Modus Ponens})$$
- $$\frac{\vdash \phi}{\vdash \forall x\phi} \quad (\text{Universal Generalisation})$$

where $\phi[t/x]$ means substituting *free occurrences* of x in ϕ with t

E.g. last rule can be used to derive $\forall x p(x) \rightarrow (q(x) \rightarrow p(x))$
(in *intermediate* steps of derivations works with open formulas)



Correctness and completeness

It can be proved that $\vdash$ is still correct and complete (Gödel)
(a formula is valid if and only if it has a proof)

Undecidability

$\vdash$ is undecidable
(consequence of undecidability of $\models$)

